Machine Learning on the web

Robot
Vision
And Beyond

Yankun (Alex) Meng, Omar Mohammed, Samar Singh, Srivaths Ravva, Yuxiang (Naxy) Dong

Problem

- Computer vision techniques can be used in order to estimate the relative locations of various human joints.
- With this joint data, the location and orientation of human body parts can be determined.
- The big picture problem we focused on this semester was how to gather and organize information obtained from pose estimation pipelines, and then how to apply them to some application.

Goals

Several challenges were encountered throughout the course of this project:

- For the initial pose tracking application:
  - There was a need for a way to track specific joints and limbs, with joint position data obtained from the Blaze pose model.
  - There was a need for a way for a user to select desired joints.

- For the yoga pose detection application:
  - We needed to train a model to recognize yoga poses
  - We needed a way to display what pose was being done by the user
  - We need a method to provide feedback to the user based on their intended pose.

Goals (cont.)

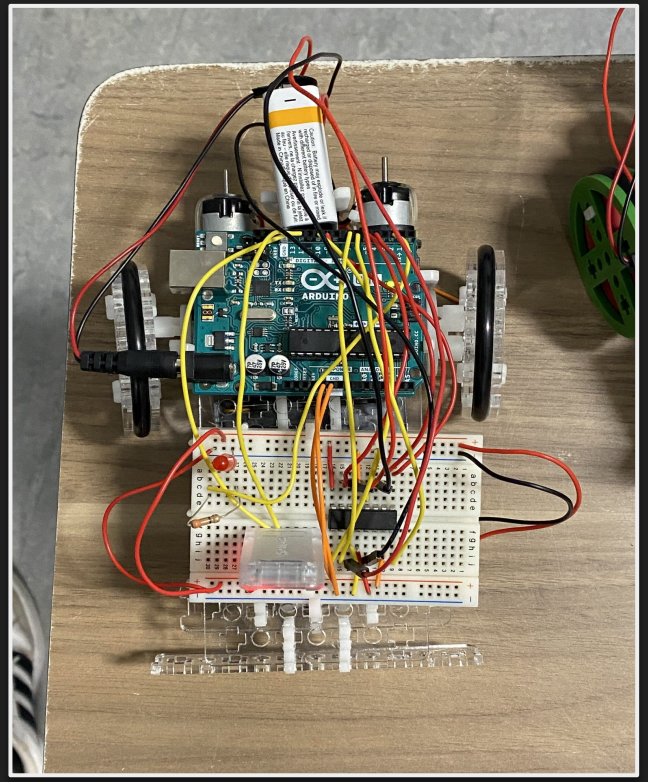
Some goals that we envisioned in response to the challenges:

- Creating a user-friendly web application that allows for the selection of specific joints or limbs.
- Visualizing the captured the movement of the tracked joints within the aforementioned web application.
- Create an application to recognize yoga poses based on a trained machine learning model, and providing feedback.
- Utilizing a user interface in order to track and export orientation data of specific limbs.
- Use joint data to obtain 2-D and 3-D limb orientation and location information.
    - Export and process that data to be applied to a mechanical representation.

Methodology

1. Getting familiar with javascript, p5, machine learning models, and other background information.
2. Training a model to recognize certain gestures, sending a command to an arduino car based on the gesture recognized.
3. Creating checkboxes using p5 and javascript.
4. Creating javascript object to store joint and limb position data.
5. Creating humanoid model on interface with selectable joints and limbs/
6. Training machine learning model with yoga poses.
7. Calculating limb position and orientation using joint data.

Controlling a robot with bluetooth using pose detection



- Goal: Blazepose was used for pose detection to move a 2 wheel drive robot through a bluetooth.

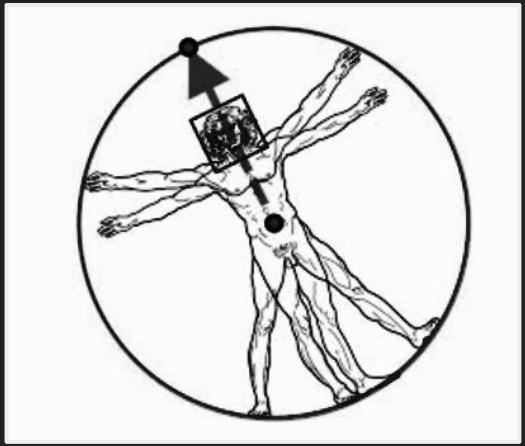
- Problems: Unable to move the robot either due to improper wiring or faulty motors.

BlazePose Model

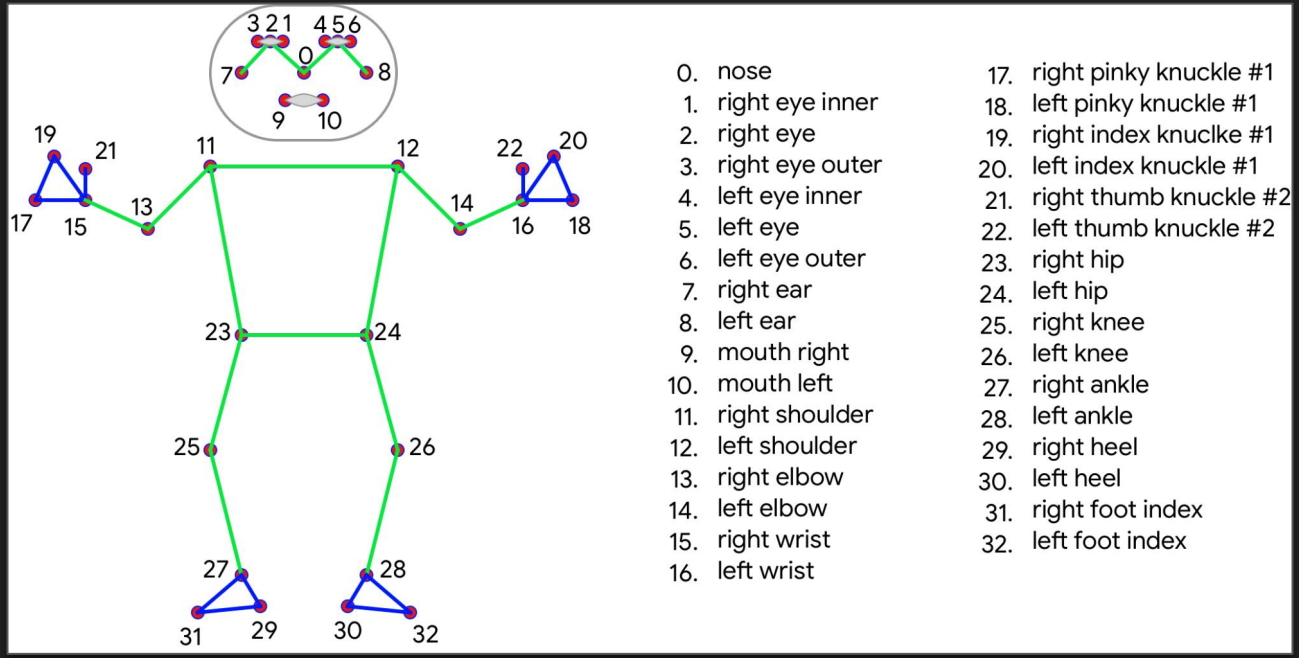
- Superset of COCO topology (17 key points), BlazeFace, and BlazePalm topologies to present a new topology of 33 human body keypoints.
- Allows for determining body semantics from pose detection alone that is consistent with face and hand models.
- If one leverages GPU inference, BlazePose achieves super-real-time performance, enabling it to run subsequent ML models, like face or hand tracking.
- lite - smallest model which trades footprint for accuracy.
- heavy - largest model intended for high accuracy, regardless of size.
- full - a middle ground between Lite and Heavy.

BlazePose Model (cont.)

- The model is built on the assumption that the face, due to its high contrast features and comparably small variations in appearance, is visible for a single person use case.
- A face detector is trained as a proxy for a pose detector. It only detects the location of a person within the frame and can not be used to identify individuals.
- Two additional virtual key points are predicted to describe the human body center, rotation, and scale as a circle.



BlazePose 33 Key Points



0. nose
1. right eye inner
2. right eye
3. right eye outer
4. left eye inner
5. left eye
6. left eye outer
7. right ear
8. left ear
9. mouth right
10. mouth left
11. right shoulder
12. left shoulder
13. right elbow
14. left elbow
15. right wrist
16. left wrist
17. right pinky knuckle #1
18. left pinky knuckle #1
19. right index knuclke #1
20. left index knuckle #1
21. right thumb knuckle #2
22. left thumb knuckle #2
23. right hip
24. left hip
25. right knee
26. left knee
27. right ankle
28. left ankle
29. right heel
30. left heel
31. right foot index
32. left foot index

Using Blazepose for tracking limbs

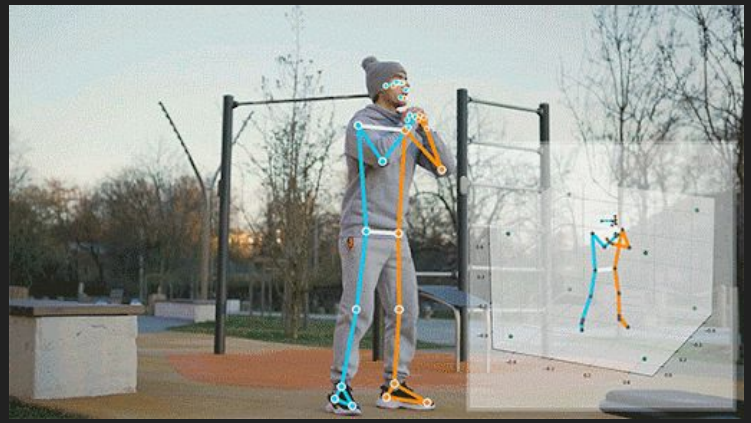
- Goal: Create a user interface to choose specific limbs to track.

```html
<label>
  <input type="checkbox" name="Body_Parts" value="Left_Upper_Arm" id="Left_Upper_Arm"/> Left Upper Arm
</label>
<label>
  <input type="checkbox" name="Body_Parts" value="Left_Lower_Arm" id="Left_Lower_Arm"/> Left Lower Arm
</label>
<label>
  <input type="checkbox" name="Body_Parts" value="Right_Upper_Arm" id="Right_Upper_Arm"/> Right Upper Arm
</label>
<label>
  <input type="checkbox" name="Body_Parts" value="Right_Lower_Arm" id="Right_Lower_Arm"/> Right Lower Arm
</label>
<label>
  <input type="checkbox" name="Body_Parts" value="Left_Upper_Leg" id="Left_Upper_Leg"/> Left Upper Leg
</label>
<label>
  <input type="checkbox" name="Body_Parts" value="Left_Lower_Leg" id="Left_Lower_Leg"/> Left Lower Leg
</label>
<label>
  <input type="checkbox" name="Body_Parts" value="Right_Upper_Leg" id="Right_Upper_Leg"/> Right Upper Leg
</label>
<label>
  <input type="checkbox" name="Body_Parts" value="Right_Lower_Leg" id="Right_Lower_Leg"/> Right Lower Leg
</label>
```

```
if(leftLowerLeg_cb.checked){
  body_parts.leftLowerArm = {joint1: {x: pose[26].x, y: pose[26].y, z: pose[26].z}, joint2: {x: pose[28].x, y: pose[28].y, z: pose[28].z}}
  drawLandmark(canvasCtx, pose[26].x * 640, pose[26].y*480, 6, 'white', 'red',2);
  drawLandmark(canvasCtx, pose[28].x*640, pose[28].y*480, 6, 'white', 'red',2);
  connectPoints(canvasCtx,pose[26].x * 640, pose[26].y*480,pose[28].x*640, pose[28].y*480)
  let j1 = [pose[26].x, pose[26].y, pose[26].z];
  let j2 = [pose[28].x, pose[28].y, pose[28].z];
  Track2.push(Homogenous_Matrix(j2,j1,0));
  canvasCtx.restore();
}

if(RightUpperArm_cb.checked){
  body_parts.leftUpperArm = {joint1: {x: pose[11].x, y: pose[11].y, z: pose[11].z}, joint2: {x: pose[13].x, y: pose[13].y, z: pose[13].z}}
  drawLandmark(canvasCtx, pose[11].x * 640, pose[11].y*480, 6, 'white', 'red',2);
  drawLandmark(canvasCtx, pose[13].x*640, pose[13].y*480, 6, 'white', 'red',2);
  connectPoints(canvasCtx,pose[11].x * 640, pose[11].y*480,pose[13].x*640, pose[13].y*480);
  let j1 = [pose[11].x, pose[11].y, pose[11].z];
  let j2 = [pose[13].x, pose[13].y, pose[13].z];
  Track2.push(Homogenous_Matrix(j2,j1,0));
  canvasCtx.restore();
}
```

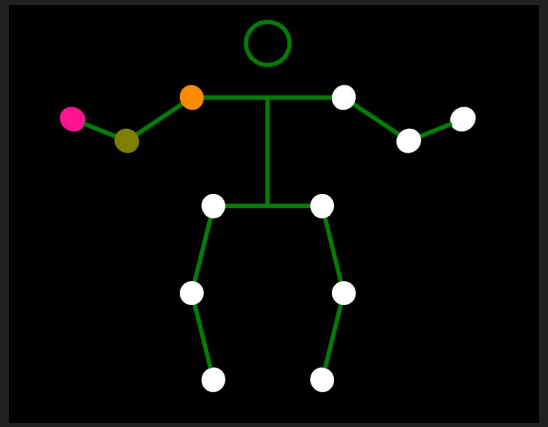
BlazePose Model 3D Key Points

- The x, y, z coordinates of a body key point can be retrieved using the keypoints 3D array.
- The x, y, z are in meter units.
- The person is modeled as if they were in a 2m x 2m x 2m cubic space.
- The range for each axis goes from -1 to 1 (therefore 2m total delta).
- The origin of this 3D space is the hip center (0, 0, 0).
- From the origin, z is positive if moving closer to the camera.

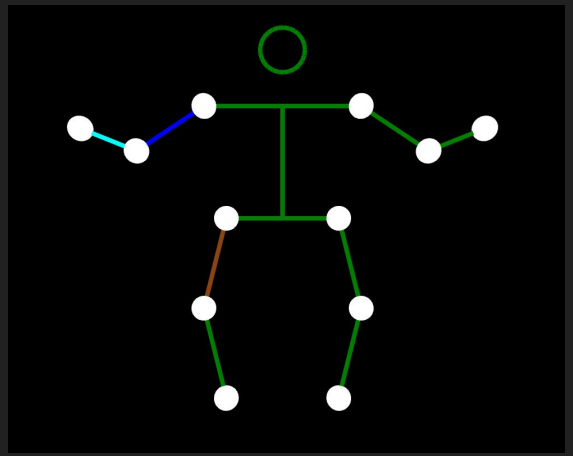


```
//draws landmarks
function drawLandmark(ctx, x, y, radius, fill, stroke, strokeWidth) {
  ctx.beginPath()
  ctx.arc(x, y, radius, 0, 2 * Math.PI, false)
  if (fill) {
    ctx.fillStyle = fill
    ctx.fill()
  }
  if (stroke) {
    ctx.lineWidth = strokeWidth
    ctx.strokeStyle = stroke
    ctx.stroke()
  }
}
//connects the landmarks
function connectPoints(ctx, x1, y1, x2, y2) {
  ctx.beginPath();
  ctx.moveTo(x1,y1);
  ctx.lineTo(x2, y2, 6);
  ctx.strokeStyle = '#000000';
  ctx.stroke()
}
```

Joint and Limb Selection Using Skeleton Model



☑ Joint Motion Capture



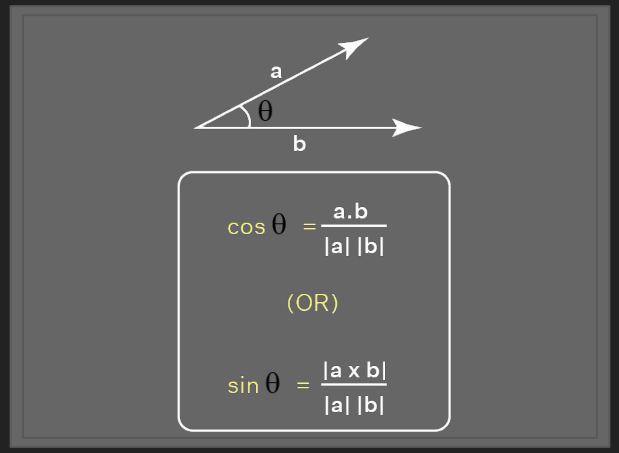
☑ Limb Motion Capture

```
p5.mousePressed = () => {
let x = p5.mouseX,
  y = p5.mouseY;
// example for head/nose
// eq: (x-centerX)**2+(y-centerY)**2 <= radius**2
// radius = 10

if (limbMotionChecked) {
  if (
    p5.dist(320, 100, x, y) + p5.dist(320, 200, x, y) <=
      p5.dist(320, 100, 320, 200) + 1 &&
    p5.dist(320, 100, x, y) + p5.dist(320, 200, x, y) >=
      p5.dist(320, 100, 320, 200) - 1
  ) {
    limbSelected.spine.selected = !limbSelected.spine.selected;
  }
```

```
function drawLimb(x1, y1, x2, y2, limb) {
p5.colorMode(p5.RGB, 255, 255, 255);
if (limb.selected) {
  p5.stroke(limb.color[0], limb.color[1], limb.color[2]);
  p5.strokeWeight(4);
  p5.line(x1, y1, x2, y2);
} else {
  p5.stroke(0, 128, 0);
  p5.strokeWeight(4);
  p5.line(x1, y1, x2, y2);
}
}
```

Obtaining Relative Limb Orientation

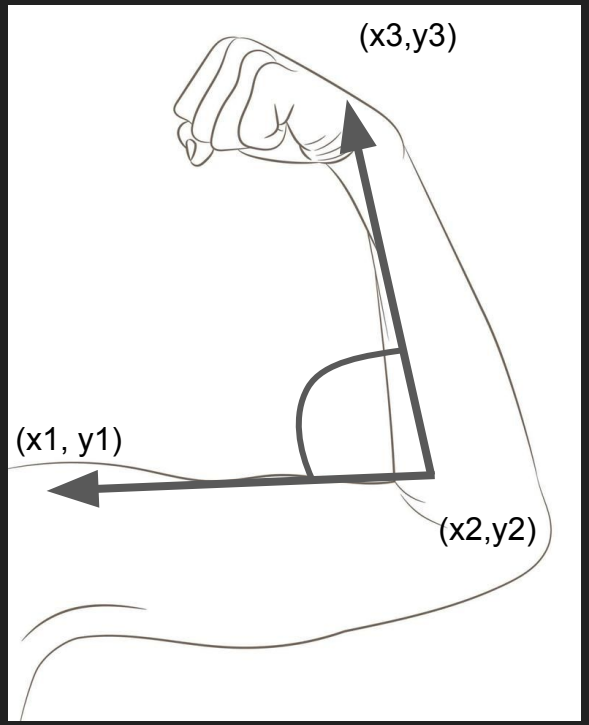
- In order to make further progress with the usage of joint position data, it was necessary to work on a method to obtain, and export the 2-D and 3-D limb orientation data.
- This is a continued work in progress, however the angle between limbs in 2-D space has been able to be found using vector algebra.
- Using the x and y coordinates obtained from the Blazepose model, the desired limbs are able to be represented as vectors and then the following equation can be applied:



Obtaining Relative Limb Orientation

Code Representation:

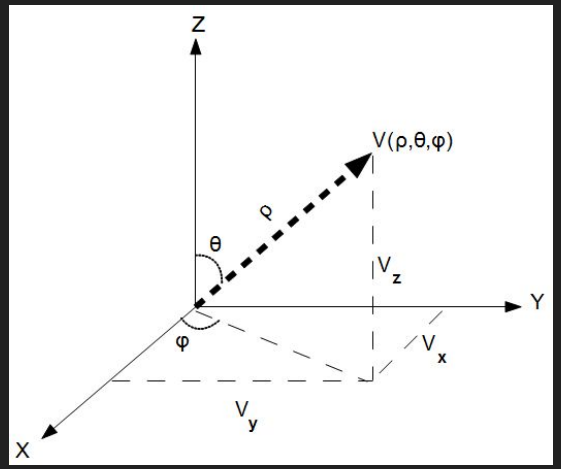
```
function getAngle(x2, y2, x1, y1, x3, y3){
  vec_a = [(x2-x1), (y2-y1)];
  vec_b = [(x3-x2), (y3-y2)];

  dotProduct = (vec_a[0] * vec_b[0])+(vec_a[1] * vec_b[1]);

  magn_a = Math.sqrt(Math.pow(vec_a[0],2) + Math.pow(vec_a[1],2));
  magn_b = Math.sqrt(Math.pow(vec_b[0],2) + Math.pow(vec_b[1],2));

  return (Math.acos(dotProduct / (magn_a*magn_b)) * 180 / Math.PI);
}
```



Obtaining Relative Limb Orientation - Plans for Next Semester

Next Steps:

- Creating interface to allow for specified limb selection
- Obtain 3-D orientation data
- Create method for exporting data obtained from movement tracking
- Create mechanical/animated representation of 2-D and 3-D data



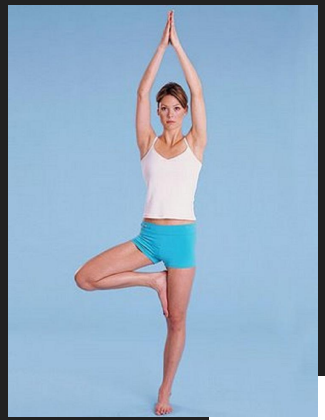
YogaNet

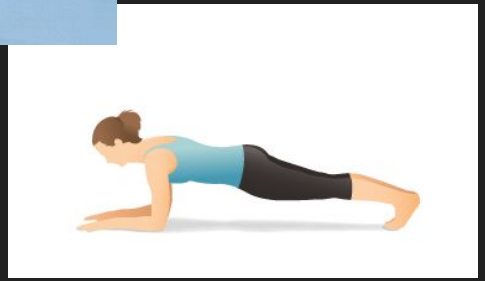
Used HTML, javascript, css, and python to build the yoga pose recognition program.

1. Data collection
2. Training and Loading the model
3. Live detection using webcam

Data Collection

- 5 different types of poses: downdog, goddess, warrior 2, tree, plank
- Use pre existing dataset of yoga poses from https://www.kaggle.com/datasets/niharika41298/yoga-poses-dataset





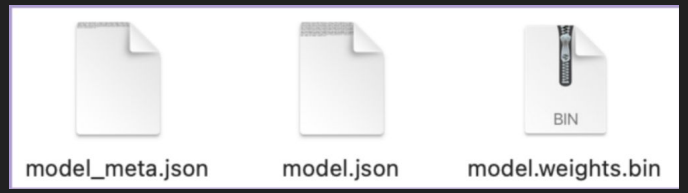
Data Collection using Python

- Extracted 33 key points into a csv file
- Columns kp (0-32) take the keypoint values from an image at each index
- Final column is the corresponding pose

1	kp 30 x	kp 30 y	kp 30 z	kp 31 x	kp 31 y	kp 31 z	kp 32 x	kp 32 y	kp 32 z	pose
2	0.493168354	0.633991718	0.239711612	0.383244485	0.69176656	0.03397619	0.3570503	0.741290867	0.21641539	downdog
3	0.477427632	0.65602845	0.228059515	0.317628771	0.693704367	0.009533337	0.321492255	0.754254282	0.208664045	downdog
4	0.395118594	0.602097571	0.271404654	0.326758385	0.68825084	0.049751982	0.28912735	0.674356103	0.256022394	downdog
5	0.146720111	0.629047215	0.253259599	0.290879667	0.714561343	0.152807206	0.042777896	0.741245747	0.214929014	downdog
6	-0.324245036	0.685757279	0.029074593	-0.196103066	0.792417049	0.23982732	-0.220849246	0.779003561	-0.024572622	downdog
7	0.464344144	0.622190356	0.200771987	0.381709665	0.707969546	0.037568018	0.335001171	0.708293319	0.171415433	downdog
8	0.502187788	0.542439818	0.252399713	0.434111774	0.644751549	0.063162059	0.388228118	0.622929454	0.232387424	downdog
9	-0.626782417	0.539017797	0.19956778	0.477440059	0.56283319	0.093547188	-0.699025512	0.53848803	0.141856432	downdog
10	0.469656765	0.619858742	0.143022016	0.383507311	0.695224643	0.018964959	0.349926054	0.695706725	0.133492827	downdog
11	0.542491078	0.58952111	0.151923507	0.443764091	0.700637341	0.039571244	0.420175076	0.706423521	0.142103091	downdog
12	0.499482125	0.544242263	0.242317498	0.46583882	0.691651642	0.057579502	0.391412258	0.655004561	0.228185892	downdog
13	0.419532835	0.638429046	0.243645653	0.319408	0.688257694	0.038670648	0.277567744	0.725258887	0.214933217	downdog
14	0.516760588	0.57379818	0.181951791	0.444083095	0.667299867	0.036613654	0.395265818	0.667158306	0.173952535	downdog
15	0.430548191	0.546987653	0.168994114	0.452609837	0.682275712	0.027594782	0.340014935	0.627615333	0.171724528	downdog
16	-0.440924883	0.640984118	0.027153267	-0.291580558	0.726880908	0.265025437	-0.29786253	0.73640132	-0.004896851	downdog
17	0.471646845	0.607578039	0.160009906	0.38437885	0.700398684	-0.01343332	0.355183482	0.692461193	0.148635045	downdog
18	0.526892722	0.521924019	0.361105829	0.45490545	0.651509345	0.117872506	0.392705828	0.619720161	0.336488634	downdog
19	0.358168691	0.667914867	0.26625526	0.277778536	0.790961683	0.018740267	0.216720253	0.749738097	0.251472414	downdog
20	-0.553103268	0.562945366	0.081793629	-0.405623138	0.61627686	0.352760255	-0.43208456	0.655262351	0.055622116	downdog
21	0.542423189	0.550936222	0.20469822	0.490672231	0.645724475	0.050067388	0.443427563	0.623411715	0.189645663	downdog
22	-0.437523007	0.697193503	-0.011760449	-0.268935204	0.705328822	0.2743783	-0.296985865	0.800596476	-0.056912094	downdog
23	0.496445566	0.595800996	0.214381322	0.381727606	0.674029768	-0.021091443	0.365500152	0.695434153	0.192396343	downdog
24	-0.381548375	0.574070632	0.158910066	-0.228390142	0.6195485	0.381680995	-0.269351393	0.645988941	0.148190781	downdog
25	0.406752586	0.562492371	0.32317692	0.391683698	0.642614841	0.107968345	0.303691387	0.667030394	0.312726349	downdog
26	0.471646845	0.607578039	0.160009906	0.38437885	0.700398684	-0.01343332	0.355183482	0.692461193	0.148635045	downdog
27	0.542423189	0.550936222	0.20469822	0.490672231	0.645724475	0.050067388	0.443427563	0.623411715	0.189645663	downdog
28	0.496445566	0.595800996	0.214381322	0.381727606	0.674029768	-0.021091443	0.365500152	0.695434153	0.192396343	downdog
29	0.526892722	0.521924019	0.361105829	0.45490545	0.651509345	0.117872506	0.392705828	0.619720161	0.336488634	downdog
30	-0.553103268	0.562945366	0.081793629	-0.405623138	0.61627686	0.352760255	-0.43208456	0.655262351	0.055622116	downdog
31	-0.437523007	0.697193503	-0.011760449	-0.268935204	0.705328822	0.2743783	-0.296985865	0.800596476	-0.056912094	downdog
32	-0.381548375	0.574070632	0.158910066	-0.228390142	0.6195485	0.381680995	-0.269351393	0.645988941	0.148190781	downdog
33	0.406752586	0.562492371	0.32317692	0.391683698	0.642614841	0.107968345	0.303691387	0.667030394	0.312726349	downdog
34	0.358168691	0.667914867	0.26625526	0.277778536	0.790961683	0.018740267	0.216720253	0.749738097	0.251472414	downdog
35	0.477775216	0.631455421	0.221521929	0.383778155	0.707968235	0.030728439	0.333964437	0.739523172	0.194427609	downdog
36	-0.508883178	0.593089104	0.05088301	-0.345061481	0.685542643	0.321548641	-0.392618477	0.705909729	0.020900642	downdog
37	-0.539521635	0.366866827	0.442608416	-0.610877752	0.316108525	0.510804772	-0.597594023	0.336844921	0.408551425	downdog

Training and Loading the Model

- Use existing data training model that uses ML5.JS from https://medium.com/aixdesign/getting-started-with-ml5-js-tutorial-part-iii-yoga-pose-detection-8b4609f122ab




```
const modelInfo = {
model: "model/model.json",
metadata: "model/model_meta.json",
weights: "model/model.weights.bin",
};

brain.load(modelInfo, brainLoaded);

let classification = false;
let poseLabel;

function brainLoaded() {
    console.log('classification ready')
    classification = true;
}
```

Live Detection using Webcam

- Feedback displayed on screen
- Text-to-speech

Results:

https://youtu.be/hdGO0vEizoc

```
function onResults(results) {
  // pose detection
  if (results.poseWorldLandmarks) {
    let inputs = []
    for(let i=0; i<33; i++){
        inputs.push(results.poseWorldLandmarks[i].x)
        inputs.push(results.poseWorldLandmarks[i].y)
        inputs.push(results.poseWorldLandmarks[i].z)
    }
    if(classification) brain.classify(inputs, gotResult);
    if(poseLabel){
      console.log(poseLabel)
      message.innerHTML = `I think your pose is "${poseLabel}".---Accurac
      if(poseLabel != prev){
        prev = poseLabel
        var msg = new SpeechSynthesisUtterance();
        msg.text = poseLabel
        window.speechSynthesis.speak(msg);
      }
    }

  }
}
```

More Plans for next semester

- Build an AR game that detects the pose the user is trying to do and recommends fixes that they can do to correct their pose.